Assignment 5 GenAI Analysis

Hunter Noravong – EECS 348 – 10/29/2026. Inherited code: Cesar Olono's Assignment 4 (Sudoku solver).

a. GenAI programs

Claude and Google Gemini.

b. How accessed

Claude: claude.ai in Google

Gemini: Google search

c. Prompt (identical for both)

Modify the attached single-file C++ Sudoku solver. Edit the existing code in place and keep its classes; do not rewrite it from scratch.

Requirements:

1. For each input file puzzleN.txt (N = 1 to 5), write the result to solutionN.txt. Each puzzle is a 9x9 grid: cells are separated by spaces and a blank cell is "_".
2. If the puzzle has no solution, or the file cannot be read, solutionN.txt must contain exactly one line: No solution found.
3. Otherwise, solutionN.txt must contain every solution, in the same format as the puzzle file, followed by one line: Solutions found: n
4. The program must be a single .cpp file that compiles with g++ on a Linux server.
5. Put a prologue comment at the top (program name, description, inputs, outputs, author, revised by, sources).

Return the complete modified file.

d. Generated code

Claude:

```
/*
 * Program name: EECS 348 Assignment 5 - Sudoku Solver (revision of Assignment 4)
 * Description: Object-oriented C++ program that solves Sudoku puzzles with a
 *              recursive depth-first search with backtracking. For each puzzle
 *              it writes every solution, or "No solution found.", to a
 *              solutionN.txt file (N = puzzle number).
```

```
 * Inputs: puzzle1.txt through puzzle5.txt (9x9 grids, "_" marks a blank cell)
 * Outputs: solution1.txt through solution5.txt. Each file holds every solution
 *          in the same format as the puzzle files, followed by the line
 *          "Solutions found: n", or the single line "No solution found."
 * Author: Cesar Olono
 * Revised by: Hunter Noravong, 10/28/2026
 * Collaborators: Cesar Olono (original Assignment 4 author)
 * Other sources: Claude wrote the base program for Cesar's Assignment 4. For Assignment 5,
 *              Claude produced the bitmask speed-up and the
 *              solutionN.txt output changes marked.
 * Creation date: 10/01/2026 (original file by Cesar Olono)
 * Revision date: 10/27/2026
 */

#include <fstream> //lets the program read the puzzle files and now also write the solution files
#include <iostream> //lets the program print to the screen
#include <sstream> //ostringstream builds "puzzleN.txt" names without needing std::to_string (works on older g++ too)
#include <string> //string for file names
#include <vector> //vectore for the grid that stores the board

// ---------------------------------------------------------------------------
// SudokuBoard: owns the 9x9 grid and knows the Sudoku rules.
// A cell value of 0 means "empty" internally; '_' is used for I/O.
// ---------------------------------------------------------------------------

class SudokuBoard {  //class SudokuBoard is one objext that holds the 9x9 puzzle and understands how to work with it
public:
    static const int SIZE = 9;    //SIZE and BLOCK are the numbers for the grid size
    static const int BLOCK = 3;   //they're useful for not having to type 9 and 3 everywhere

    // the grid starts empty (all 0s) and the three mask lists start with no digits used.
    SudokuBoard() : grid_(SIZE, std::vector<int>(SIZE, 0)), rowMask_(SIZE, 0), colMask_(SIZE, 0), boxMask_(SIZE, 0) {}

    // Load a puzzle from a file. Returns false if the file can't be opened, has characters other than digits,
    // _ and whitespace, or doesn't have exactly 81 cells.
```

```
    bool loadFromFile(const std::string& filename) { //reads a puzzle file into the grid one at a time,
    std::ifstream in(filename.c_str());             //and returns false if the file is bad or doesn't have 81 cells
    if (!in) {                      // [Cesar] the file could not be opened
        return false;               // [Cesar] report failure to the caller
    }

    int count = 0;                  // [Cesar] how many cells have been read so far
    char ch;                        // [Cesar] the character currently being read
    while (in.get(ch)) {            // [Cesar] read the file one character at a time until it ends
        if (ch == ' ' || ch == '\n' || ch == '\r' || ch == '\t') {
            continue;               // whitespace between cells is fine
        }
        if (count >= SIZE * SIZE) {
            return false;           // more than 81 cells in the file
        }
        if (ch == '_') {
            set(count / SIZE, count % SIZE, 0);           // [Revised] now goes through set() so the masks
stay correct; '_' = empty
        } else if (ch >= '1' && ch <= '9') {
            set(count / SIZE, count % SIZE, ch - '0');    // [Revised] same: turn the character into a digit
and store it via set()
        } else {
            return false;           // a character that isn't a cell
        }
        ++count;                    // [Cesar] one more cell has been stored
    }
    return count == SIZE * SIZE;    // [Cesar] valid only if we read exactly 81 cells
}

// Puts a number into one cell AND keeps the row/column/box digit lists (bitmasks) in sync.
// Bit d of a mask is 1 when digit d is already used in that row/column/box.
void set(int row, int col, int value) {
    int box = (row / BLOCK) * BLOCK + col / BLOCK;  // which of the 9 boxes this cell is in (0-8)
    int old = grid_[row][col];                      // the digit that is there right now (0 if empty)
    if (old != 0) {                             // if a digit is being replaced or erased...
        int clear = ~(1 << old);                // ...build a mask that has a 0 only in that digit's bit
        rowMask_[row] &= clear;                 // that digit is no longer used in the row
        colMask_[col] &= clear;                 // ...nor in the column
```

```
            boxMask_[box] &= clear;                  // ...nor in the box
        }
        grid_[row][col] = value;                     // store the new value in the grid
        if (value != 0) {                            // if a real digit was placed (not an erase)...
            int bit = 1 << value;                    // ...get that digit's bit
            rowMask_[row] |= bit;                    // mark it used in the row
            colMask_[col] |= bit;                    // ...the column
            boxMask_[box] |= bit;                    // ...and the box
        }
    }

    // Returns a bitmask whose bit d is 1 if digit d (1-9) could legally go at (row, col).
    // One OR and one AND replaces the 27-cell scan the Assignment 4 version did for every digit.
    int candidateMask(int row, int col) const {
        int used = rowMask_[row] | colMask_[col] | boxMask_[(row / BLOCK) * BLOCK + col / BLOCK]; // every digit already blocked for this cell
        return ~used & 0x3FE;                        // flip it, keep only bits 1..9 (0x3FE = binary 1111111110)
    }

    // Is 'digit' allowed at (row, col)? Checks row, column and 3x3 block.
    bool isValidPlacement(int row, int col, int digit) const { //checks that a digit isn't already in that cell's row, column, or block
        return ((candidateMask(row, col) >> digit) & 1) != 0;  // true if the digit's bit is set in the candidate mask
    }

    // Count how many digits (1-9) could legally go in this cell.
    int countCandidates(int row, int col) const { //counts how many digits from 1 to 9 could legally go in an empty cell
        return __builtin_popcount(candidateMask(row, col));    // popcount = number of 1 bits = number of legal digits (GCC built-in)
    }
    // Find the empty cell with the fewest valid digits. Returns false if
    // there are no empty cells.
    bool findBestEmpty(int& row, int& col) const { //finds the empty cell with the least digis so the solver guesses athe the most force cell
        int bestCount = 10;  // higher than any real count (max is 9)
        bool found = false;  // [Cesar] stays false if the board has no empty cells
        for (int r = 0; r < SIZE; ++r) {             // [Cesar] go through every row
```

```
        for (int c = 0; c < SIZE; ++c) {        // [Cesar] and every column
            if (grid_[r][c] == 0) {             // [Cesar] only empty cells matter
                int count = countCandidates(r, c);  // [Cesar] how many digits fit here
                if (count < bestCount) {        // [Cesar] fewer than the best so far?
                    bestCount = count;          // [Cesar] remember the new best count
                    row = r;                    // [Cesar] and where it is
                    col = c;
                    found = true;               // [Cesar] we have at least one empty cell
                    if (count <= 1) return true; // [Claude] 0 = dead end, 1 = forced move; nothing can beat these so stop looking
                }
            }
        }
    }
    return found;                               // [Cesar] false means the board is full
  }
  // Do the pre-filled cells already break a rule? (e.g. two 5s in a row)
  bool givensAreConsistent() { //checks that the digits already in the puzzle don't break a rule before solving starts
    // [Revised] The masks can't be used here: if a row held two 5s the mask would only record one of them.
    // So this version keeps its own scratch masks and flags a digit the moment it is seen twice.
    int rows[SIZE], cols[SIZE], boxes[SIZE];    // digits already seen in each row/column/box during this check
    for (int i = 0; i < SIZE; ++i) {            // start all of them empty
        rows[i] = 0;
        cols[i] = 0;
        boxes[i] = 0;
    }
    for (int r = 0; r < SIZE; ++r) {            // [Cesar] look at every cell
        for (int c = 0; c < SIZE; ++c) {
            int digit = grid_[r][c];            // [Cesar] the given digit (0 = blank)
            if (digit == 0) continue;           // [Cesar] blanks can't break a rule
            int bit = 1 << digit;               // this digit's bit
            int box = (r / BLOCK) * BLOCK + c / BLOCK; // [Claude] which box the cell is in
            if ((rows[r] & bit) || (cols[c] & bit) || (boxes[box] & bit)) {
                return false;                   // already seen in the row, column or box = rule broken
            }
```

```
            rows[r] |= bit;                 // otherwise record it and keep checking
            cols[c] |= bit;
            boxes[box] |= bit;
        }
    }
    return true;                            // [Cesar] no duplicates anywhere
}

void print(std::ostream& out) const { //prints the grid with _ for empty cells
    for (int r = 0; r < SIZE; ++r) {            // [Cesar] one printed line per row
        for (int c = 0; c < SIZE; ++c) {        // [Cesar] cells in the row
            if (c > 0) out << ' ';              // [Cesar] single space between cells
            if (grid_[r][c] == 0) out << '_';   // [Cesar] blank cell
            else out << grid_[r][c];            // [Cesar] filled cell
        }
        out << '\n';                            // [Cesar] end of the row
    }
}

private:
    std::vector<std::vector<int> > grid_;           // [Cesar] the 9x9 board, 0 = empty
    std::vector<int> rowMask_, colMask_, boxMask_;  // digits used per row / column / box (bit d = digit d)
};

// ----------------------------------------------------------------------------
// SudokuSolver: recursive DFS + backtracking. Writes every solution as it is found.
// ----------------------------------------------------------------------------
class SudokuSolver { //does the recursive search trying digits and backs up when stuck and writes every solution as it finds them
public:
    SudokuSolver() : solutionCount_(0), out_(0) {}  // [Revised] out_ starts as "no stream yet"

    // Solve a copy of the given puzzle. Each solution is written to 'out' as soon
    // as it is found. Returns how many solutions were found.
    int solveAll(const SudokuBoard& puzzle, std::ostream& out) { // [Revised] now takes the stream to write solutions to
        solutionCount_ = 0;                     // [Cesar] start counting from zero
        out_ = &out;                            // [Revised] remember where solutions should be written
```

```
        SudokuBoard working = puzzle;              // [Cesar] solve a copy so the original puzzle is untouched
        if (working.givensAreConsistent()) {       // [Cesar] skip the search if the givens already break a rule
            solve(working);                        // [Cesar] run the recursive search
        }
        return solutionCount_;                     // [Cesar] how many were found
    }

private:
    int solutionCount_;                            // [Cesar] number of complete solutions found so far
    std::ostream* out_;                            // [Revised] where solutions are written (the solutionN.txt file)

    void solve(SudokuBoard& board) {
        int row, col;                              // [Cesar] the cell chosen for this step
        if (!board.findBestEmpty(row, col)) {
            ++solutionCount_;  // no empty cells: a full solution
            if (solutionCount_ > 1) *out_ << "\n";  // [Revised] blank line between solutions (not before the
first)
            board.print(*out_);                    // [Revised] the grid only, same format as the puzzle files
            return;
        }

        for (int digit = 1; digit <= 9; ++digit) {
            if (board.isValidPlacement(row, col, digit)) {
                board.set(row, col, digit);  // try it
                solve(board);                // recurse
                board.set(row, col, 0);      // undo (backtrack)
            }
        }
        // Keep going after a success so that *all* solutions are found.
    }
};

// ----------------------------------------------------------------------------
// PuzzleRunner: handles files and output formatting for each puzzle.
// ----------------------------------------------------------------------------
class PuzzleRunner { //loads one puzzle file, runs the solver and writes the result to solutionN.txt
public:
```

```
   void run(int number) {                    // [Revised] takes the puzzle number so both file names can be built
      std::ostringstream inName, outName;    // builders for the two file names
      inName << "puzzle" << number << ".txt";     // e.g. puzzle3.txt
      outName << "solution" << number << ".txt";  // e.g. solution3.txt
      std::string filename = inName.str();       // [Revised] input file name as a string

      std::ofstream outFile(outName.str().c_str()); // [Revised] open (and overwrite) the solution file
      if (!outFile) {                            // [Revised] could not create the solution file
         std::cout << "Error: could not create " << outName.str() << "\n";
         return;                                 // nothing more can be done for this puzzle
      }

      SudokuBoard puzzle;                        // [Cesar] the board that will hold the puzzle
      if (!puzzle.loadFromFile(filename)) {      // [Cesar] bad or missing file
         std::cout << "Error: could not read a valid puzzle from " << filename << "\n";
         outFile << "No solution found.\n";      // [Revised] an unreadable puzzle has no solution, so the file says so
         return;
      }

      SudokuSolver solver;                       // [Cesar] a fresh solver for this puzzle
      int found = solver.solveAll(puzzle, outFile); // [Revised] solutions are written straight into the file

      if (found == 0) {
         outFile << "No solution found.\n";      // [Revised] required exact wording, with the period
      } else {
         outFile << "\nSolutions found: " << found << "\n"; // [Revised] required summary line, after a blank separator line
      }
      std::cout << filename << ": " << found << " solution(s) written to " << outName.str() << "\n"; // [Revised] screen progress line
   }
};

int main() { //runs the runner on puzzle 1 through 5
   PuzzleRunner runner;                          // [Cesar] one runner handles all five puzzles
   for (int i = 1; i <= 5; ++i) {                // [Cesar] puzzle numbers 1 to 5
```

```
        runner.run(i);                          // [Revised] pass the number instead of building the file name here
    }
    return 0;                                   // [Cesar] normal exit
}
```

Gemini:

```
/*
 * Program name: EECS 348 Assignment 5 - Sudoku Solver
 * Description: Object-oriented C++ program that solves Sudoku puzzles with a
 *              recursive depth-first search with backtracking. It writes
 *              every solution for each puzzle to solutionN.txt, or "No solution found."
 * Inputs: puzzle1.txt through puzzle5.txt (9x9 grids, "_" marks a blank cell)
 * Outputs: solution1.txt through solution5.txt containing solutions or "No solution found."
 * Author: Cesar Olono
 * Revised by: Hunter Noravong
 * Revision date: 10/27/2026
 * Other sources: Claude (base program architecture)
 */

#include <fstream>
#include <iostream>
#include <string>
#include <vector>

class SudokuBoard {
public:
    static const int SIZE = 9;
    static const int BLOCK = 3;

    SudokuBoard() : grid_(SIZE, std::vector<int>(SIZE, 0)) {}

    bool loadFromFile(const std::string& filename) {
        std::ifstream in(filename.c_str());
        if (!in) {
            return false;
        }
```

```
    int count = 0;
    char ch;
    while (in.get(ch)) {
        if (ch == ' ' || ch == '\n' || ch == '\r' || ch == '\t') {
            continue;
        }
        if (count >= SIZE * SIZE) {
            return false;
        }
        if (ch == '_') {
            grid_[count / SIZE][count % SIZE] = 0;
        } else if (ch >= '1' && ch <= '9') {
            grid_[count / SIZE][count % SIZE] = ch - '0';
        } else {
            return false;
        }
        ++count;
    }
    return count == SIZE * SIZE;
}

void set(int row, int col, int value) { grid_[row][col] = value; }

bool isValidPlacement(int row, int col, int digit) const {
    for (int i = 0; i < SIZE; ++i) {
        if (grid_[row][i] == digit) return false;
        if (grid_[i][col] == digit) return false;
    }
    int blockRow = (row / BLOCK) * BLOCK;
    int blockCol = (col / BLOCK) * BLOCK;
    for (int r = blockRow; r < blockRow + BLOCK; ++r) {
        for (int c = blockCol; c < blockCol + BLOCK; ++c) {
            if (grid_[r][c] == digit) return false;
        }
    }
    return true;
}
```

```
int countCandidates(int row, int col) const {
    int count = 0;
    for (int digit = 1; digit <= 9; ++digit) {
        if (isValidPlacement(row, col, digit)) {
            ++count;
        }
    }
    return count;
}

bool findBestEmpty(int& row, int& col) const {
    int bestCount = 10;
    bool found = false;
    for (int r = 0; r < SIZE; ++r) {
        for (int c = 0; c < SIZE; ++c) {
            if (grid_[r][c] == 0) {
                int count = countCandidates(r, c);
                if (count < bestCount) {
                    bestCount = count;
                    row = r;
                    col = c;
                    found = true;
                }
            }
        }
    }
    return found;
}

bool givensAreConsistent() {
    for (int r = 0; r < SIZE; ++r) {
        for (int c = 0; c < SIZE; ++c) {
            int digit = grid_[r][c];
            if (digit == 0) continue;
            grid_[r][c] = 0;
            bool ok = isValidPlacement(r, c, digit);
```

```
                grid_[r][c] = digit;
                if (!ok) return false;
            }
        }
        return true;
    }

    void print(std::ostream& out) const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                if (c > 0) out << ' ';
                if (grid_[r][c] == 0) out << '_';
                else out << grid_[r][c];
            }
            out << '\n';
        }
    }

private:
    std::vector<std::vector<int> > grid_;
};

class SudokuSolver {
public:
    SudokuSolver() : solutionCount_(0) {}

    int solveAll(const SudokuBoard& puzzle, std::ostream& out) {
        solutionCount_ = 0;
        SudokuBoard working = puzzle;
        if (working.givensAreConsistent()) {
            solve(working, out);
        }
        return solutionCount_;
    }

private:
    int solutionCount_;
```

```
    void solve(SudokuBoard& board, std::ostream& out) {
        int row, col;
        if (!board.findBestEmpty(row, col)) {
            ++solutionCount_;
            board.print(out);
            out << '\n';
            return;
        }

        for (int digit = 1; digit <= 9; ++digit) {
            if (board.isValidPlacement(row, col, digit)) {
                board.set(row, col, digit);
                solve(board, out);
                board.set(row, col, 0);
            }
        }
    }
};

class PuzzleRunner {
public:
    void run(const std::string& inputFilename, const std::string& outputFilename) {
        SudokuBoard puzzle;
        if (!puzzle.loadFromFile(inputFilename)) {
            std::ofstream out(outputFilename.c_str());
            out << "No solution found.\n";
            return;
        }

        std::string tempOutput = "";
        // Use a stringstream to capture solutions first
        // To strictly match format: solutions followed by "Solutions found: n"
        // or a single line "No solution found."

        SudokuSolver solver;
        // Let's capture output in a temporary file or string buffer
```

```
        std::ofstream tempOut("temp_sol.txt");
        int found = solver.solveAll(puzzle, tempOut);
        tempOut.close();

        std::ofstream out(outputFilename.c_str());
        if (found == 0) {
            out << "No solution found.\n";
        } else {
            // Read back and write solutions
            std::ifstream tempIn("temp_sol.txt");
            std::string line;
            while (std::getline(tempIn, line)) {
                out << line << "\n";
            }
            tempIn.close();
            out << "Solutions found: " << found << "\n";
        }
        out.close();
    }
};

int main() {
    PuzzleRunner runner;
    for (int i = 1; i <= 5; ++i) {
        std::string num = std::to_string(i);
        runner.run("puzzle" + num + ".txt", "solution" + num + ".txt");
    }
    return 0;
}
```

e. Analysis

Correctness

- Gemini: Compiles with no warnings. Output is identical to Claude's on all five test puzzles. Leaves an extra file, temp_sol.txt, in the folder, which the assignment says can cost a zero. Fails to compile under -std=c++98 (std::to_string).
- Claude: Compiles with no warnings under C++98, C++11 and C++17. Output is identical to

Gemini's on all five tests. Creates only the solutionN.txt files.

Execution time

- Gemini: About 1.3–1.4 s on the hard puzzle set. The legality check scans 27 cells per digit, repeated for every empty cell. Each solution is also written to a temp file and then copied again into the solution file.
- Claude: About 0.05 s on the same set (roughly 27x faster). The legality check is one bitmask operation.

Space complexity

- Gemini: O(1) board, recursion depth up to 81. The temp file on disk holds every solution, so it grows with the number of solutions.
- Claude: O(1) board plus three 9-int bitmask vectors; recursion depth up to 81. Solutions stream straight to the output file.

Maintainability

- Gemini: Keeps Cesar's three classes. Very few comments. The prologue is incomplete: the author is still Cesar, the revised-by name is a placeholder, and it lists sources that were not used. Has an unused variable and leftover comments in run(); "temp_sol.txt" is hard-coded.
- Claude: Keeps Cesar's three classes. Full prologue, a comment on every line, and each block tagged with its author. run() builds both file names from the puzzle number. Added complexity: the masks stay correct only if every change goes through set(), and __builtin_popcount is GCC-specific.

f. GenAI selected and why

I used Claude as the basis. Both programs were equally correct on my tests, but Claude's was just a little bit faster, left no extra files in the folder, compiled under older C++ standards, and came with the per-line comments and authorship tags the rubric requires. Gemini's approach would need all of those fixed.

g. What I improved

• **Correctness:** results go to solutionN.txt in the required format which is all grids, then "Solutions found: n", or exactly "No solution found.". The duplicate-givens check uses its own scratch masks so two equal digits in a row, column or box are always caught. An unreadable puzzle also writes "No solution found."

• **Execution time:** each row, column and box keeps a bitmask of used digits, so the legality check is O(1) instead of a 27-cell scan, and the search stops looking as soon as it finds a cell with 0 or 1 candidates. About 2.19 s down to about 0.1 s on my hard test puzzle.

• **Space complexity:** three extra 9-int vectors; still O(1) extra space.

• **Maintainability:** kept Cesar's class design, made PuzzleRunner::run take the puzzle number, replaced std::to_string so it also builds on older g++, and added the full prologue, per-line comments and

authorship tags.
I updated the prologue with my name and the sources, compiled the program on cycle1, and ran it.
Here is the real time I ran on cycle1:

```
real    0m0.014s
user    0m0.000s
sys     0m0.004s
```